

Връзки между таблици. Външни ключове.

Какво е ключ в таблица? – В една база данни информацията рядко е само в една таблица.

Обикновено данните са разделени в няколко таблици, които описват различни обекти – например клиенти, коли, поръчки. За да може тези таблици да работят заедно, между тях се създават **връзки**.

- Първичен ключ (PRIMARY KEY) – колона, която:
 - идентифицира реда еднозначно
 - не се повтаря
 - не е NULL

Първичният ключ е поле, което разпознава всеки ред в таблицата.

`Id INT AUTO_INCREMENT PRIMARY KEY`

- Външен ключ (FOREIGN KEY) – колона, която:
 - сочи към първичния ключ на друга таблица
 - създава връзка между таблиците
 - гарантира коректност на данните

Външният ключ прави връзка между две таблици.

Най-често използваната връзка е 1 към много (1 : M)

Пример от реалния живот:

Един клиент → много поръчки

Една категория → много коли

Един учител → много ученици

Един ред от първата таблица може да отговаря на много редове от втората.

Нека имаме следната таблица с клиенти:

```
CREATE TABLE Customers (  
  Id INT AUTO_INCREMENT PRIMARY KEY,  
  FullName VARCHAR(100) NOT NULL,  
  City VARCHAR(50)  
) ENGINE=InnoDB;
```

Id – първичен ключ, всеки клиент има уникален номер

Нека имаме и втора таблица с поръчки:

```
CREATE TABLE Orders (  
    Id INT AUTO_INCREMENT PRIMARY KEY,  
    CustomerId INT NOT NULL,  
    OrderDate DATE NOT NULL,  
    FOREIGN KEY (CustomerId) REFERENCES Customers(Id)  
) ENGINE=InnoDB;
```

CustomerId е външен ключ, той сочи към Customers.Id

MySQL казва: CustomerId в Orders сочи към Id в Customers.

Поръчка без клиент не може да съществува.

Затова не може да има поръчка за несъществуващ клиент.

Външният ключ гарантира, че всяка поръчка е свързана със съществуващ клиент.

Нека въведем данни:

1. Вмъкваме клиенти

```
INSERT INTO Customers (FullName, City)  
VALUES ('Ivan Petrov', 'Sofia'),  
       ('Maria Georgieva', 'Plovdiv'),  
       ('Georgi Ivanov', 'Varna');
```

Таблицата вече има:

- клиент с Id = 1
- клиент с Id = 2
- клиент с Id = 3

2. Вмъкваме поръчки

```
INSERT INTO Orders (CustomerId, OrderDate)  
VALUES (1, '2026-02-01'),  
       (1, '2026-02-05'),  
       (2, '2026-02-06');
```

Ще мине ли?

```
INSERT INTO Orders (CustomerId, OrderDate)  
VALUES (10, '2026-02-07');
```

Ще мине ли?

Какво е ENGINE = InnoDB?

ENGINE = InnoDB указва, че таблицата използва **InnoDB storage engine** – механизма в MySQL, който определя как се съхраняват данните и какви функционалности се поддържат.

InnoDB поддържа:

- **FOREIGN KEY** (външни ключове)
- **транзакции (COMMIT / ROLLBACK)**
- **референтна цялост**
- **заклучване на ниво ред (row-level locking)**

Поради това InnoDB е **препоръчваният engine** за релационни бази данни.

InnoDB е storage engine по подразбиране от MySQL 5.5.

Това означава, че ако **не укажем ENGINE=InnoDB** при CREATE TABLE, MySQL **автоматично създава таблицата като InnoDB.**

Връзка 1:1 (едно към едно)

На един запис от първата таблица съответства точно един запис от втората таблица и обратно

Използва се когато:

- разделяме данните на „основни“ и „допълнителни“
- искаме да пазим подробности в отделна таблица
- имаме „разширение“ на обект

Как се реализира 1:1 в MySQL?

Най-сигурният и ясен начин е във втората таблица да имаме колона, която е **едновременно PRIMARY KEY и FOREIGN KEY** към първата.

Така системата гарантира „само един ред“.

Нека имаме две таблици Customers и CustomerDetails:

```
CREATE TABLE Customers (  
  Id INT AUTO_INCREMENT PRIMARY KEY,  
  FullName VARCHAR(100) NOT NULL  
) ENGINE=InnoDB;
```

```
CREATE TABLE CustomerDetails (  
  CustomerId INT PRIMARY KEY,
```

```
Address VARCHAR(120),  
Phone VARCHAR(20),  
FOREIGN KEY (CustomerId) REFERENCES Customers(Id)  
) ENGINE=InnoDB;
```

CustomerDetails.CustomerId е PRIMARY KEY и не може да се повтаря. Същевременно е FOREIGN KEY, трябва да съществува в Customers. Следователно един клиент може да има **най-много един** ред в CustomerDetails.

Нека въведем данни:

```
INSERT INTO Customers (FullName)  
VALUES ('Ivan Petrov'),  
      ('Maria Georgieva');
```

Да приемем, че Id-тата са 1 и 2.

```
INSERT INTO CustomerDetails (CustomerId, Address, Phone)  
VALUES (1, 'Sofia, ul. A 10', '0888123456'),  
      (2, 'Plovdiv, ul. B 5', '0899123456');
```

Ще мине ли?

```
INSERT INTO CustomerDetails (CustomerId, Address, Phone)  
VALUES (99, 'Varna', '0877000000');
```

Ще мине ли?

```
INSERT INTO CustomerDetails (CustomerId, Address, Phone)  
VALUES (1, 'Other address', '000');
```

- FK пази референцията
- PK пази „само един запис“

Връзка М:М (много към много)

Един запис от първата таблица може да е свързан с много записи от втората таблица и един запис от втората таблица може да е свързан с много записи от първата таблица.

Примери:

Един ученик изучава много предмети и един предмет се изучава от много ученици

Една книга има няколко автора и един автор има няколко книги.

Как се реализира М:М в релационна БД?

- М:М не се прави директно с един FK.
- Решението е междинна таблица, която съдържа FK към А и FK към В
- често комбиниран PRIMARY KEY (за да няма повторения)

Нека имаме две таблици Cars и Extras:

```
CREATE TABLE Cars (  
    Id INT AUTO_INCREMENT PRIMARY KEY,  
    PlateNumber VARCHAR(20) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

```
CREATE TABLE Extras (  
    Id INT AUTO_INCREMENT PRIMARY KEY,  
    ExtraName VARCHAR(50) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

Междинна таблица CarExtras (двата FK + комбиниран PK)

```
CREATE TABLE CarExtras (  
    CarId INT NOT NULL,  
    ExtraId INT NOT NULL,  
    PRIMARY KEY (CarId, ExtraId),  
    FOREIGN KEY (CarId) REFERENCES Cars(Id),  
    FOREIGN KEY (ExtraId) REFERENCES Extras(Id)  
) ENGINE=InnoDB;
```

- CarExtras пази връзките „коя кола има коя екстра“
- една кола може да се среща много пъти с различни ExtraId
- една екстра може да се среща много пъти с различни CarId
- комбинираният PK не позволява една и съща двойка да се дублира

Нека въведем данни:

```
INSERT INTO Cars (PlateNumber)  
VALUES ('CA1234AB'),
```

```
('CA5678CD');
```

Да приемем Id-та 1 и 2.

```
INSERT INTO Extras (ExtraName)
VALUES ('GPS'),
      ('AirConditioning'),
      ('ChildSeat');
```

Да приемем Id-та 1, 2, 3.

Връзки в междинната таблица

```
INSERT INTO CarExtras (CarId, ExtraId)
VALUES (1, 1), -- кола 1 има GPS
      (1, 2), -- кола 1 има климатик
      (2, 2), -- кола 2 има климатик
      (2, 3); -- кола 2 има детско столче
```

```
INSERT INTO CarExtras (CarId, ExtraId) VALUES (99, 1);
```

Ще мине ли?

```
INSERT INTO CarExtras (CarId, ExtraId) VALUES (1, 99);
```

Ще мине ли?

```
INSERT INTO CarExtras (CarId, ExtraId) VALUES (1, 1);
```

Ще мине ли?

- FK пазят референциите
- комбинираният PK пази от повторения

Каскадни действия (Cascades) в MySQL

Каскадите са правила, които определят какво се случва във външния ключ, когато в референтната таблица се изпълни:

- DELETE
- UPDATE

Каскадите се отнасят само за FOREIGN KEY.

Къде се пишат каскадите?

Каскадите се пишат **в дефиницията на FOREIGN KEY**, при CREATE TABLE (или ALTER TABLE).

Общ синтаксис:

FOREIGN KEY (fk_column)

REFERENCES ParentTable(pk_column)

ON DELETE <действие>

ON UPDATE <действие>

RESTRICT / NO ACTION (по подразбиране)

За какво се използва?

- когато **не искаме автоматични промени**
- когато данните трябва да се запазят строго консистентни

Какво прави?

- **забранява** DELETE или UPDATE на ключа, ако има референции

Пример:

FOREIGN KEY (CustomerId) REFERENCES Customers(Id)

ON DELETE RESTRICT

ON UPDATE RESTRICT

CASCADE

За какво се използва?

- когато редовете във външната таблица **нямат смисъл без референтния ред**

Какво прави?

- при DELETE изтрива свързаните редове
- при UPDATE обновява FK стойностите

Пример:

FOREIGN KEY (CustomerId) REFERENCES Customers(Id)

ON DELETE CASCADE

ON UPDATE CASCADE

SET NULL

За какво се използва?

- когато искаме да **запазим редовете**, но да премахнем връзката

Какво прави?

- при DELETE FK колоната става NULL
- при UPDATE FK колоната става NULL

FK колоната трябва да допуска NULL

Пример:

FOREIGN KEY (CustomerId) REFERENCES Customers(Id)

ON DELETE SET NULL

ON UPDATE SET NULL

Каскадите винаги са към FOREIGN KEY, а действието се задейства от промяна в референтния PRIMARY KEY.

Пример (Customers → Orders)

```
CREATE TABLE Orders (  
    Id INT PRIMARY KEY,  
    CustomerId INT,  
    FOREIGN KEY (CustomerId) REFERENCES Customers(Id)  
    ON DELETE CASCADE  
    ON UPDATE RESTRICT  
);
```

Какво означава това:

- ако изтрием клиент неговите поръчки се трият
- ако се опитаме да променим Customers.Id операцията се отказва